

CLAUDE CODE · HANDBOOK

The handbook you actually finish.

52 pages. Free. Works for developers and knowledge workers. Hooks, sub-agents, skills, plan-first development, and the audit loop that keeps it all from rotting over time.

FOREWORD

Why this exists

If you've used Claude Code for more than two weeks, you've probably noticed a pattern: the first 10 minutes of every session are spent re-explaining your stack, your conventions, the thing you fixed yesterday. You correct the same drift over and over. You discover that Claude wrote a destructive command into your `.env` file. You stop trusting it for anything serious.

None of this is Claude's fault. It's an artifact of running an LLM with no project context, no guardrails, and no memory. This handbook fixes that — not with a framework or a heavy abstraction, but with a small set of files that live in your repo and turn Claude Code into a reproducible, opinionated collaborator.

By the end of the book you'll have:

- Permanent project context (a `CLAUDE.md` loaded every session)
- Defensive guardrails that block destructive commands
- Specialized sub-agents for code audit, security, and your own domain
- A skill system for domain knowledge that survives re-clones
- A plan-first workflow with a "staff engineer" reviewer baked in
- An audit loop that catches drift before you do

The whole setup takes about 30 minutes to bootstrap and a few hours to fully tune. The compound payoff is measured in days saved per quarter.

This is V2 of the handbook. What's new: full chapters on skills (the way Anthropic structures them), the plan-first workflow inspired by Claude Code's creator Boris Cherny, the audit loop, parallel worktrees, and the four-locations memory matrix. The two case studies at the end (a solo developer and a knowledge worker) show the setup in real life.

FOR WHO

This book is for you if...

Yes, this is for you

- **A developer** (any stack, any seniority) who uses Claude Code daily or wants to.
- **A knowledge worker** — writer, researcher, analyst — who uses Claude Code to write, research, or generate content and wants a stable, repeatable setup instead of starting from scratch each session.
- **A solo founder or small-team lead** who wants to scale their own output without scaling headcount.
- **A team that wants a shared, opinionated baseline** so everyone configures Claude the same way and the setup itself is reviewable in PRs.
- **A non-native English speaker** — the entire book is available in French with the same depth, no shortcuts.

This is NOT for you (yet) if...

- **You've never opened a terminal.** Start with the official Anthropic docs first — code.claude.com/docs. Come back when you've installed Claude Code and had your first session.
- **You want a 500-page reference.** This is a focused 52-page book you can finish in two hours. If you want encyclopedic coverage with 181 skill templates and 271 quiz questions, the Ultimate Guide by Florian Bruniaux is excellent.
- **You expect this to write code *for* you.** This book is about *configuring Claude Code*, not replacing yourself with it. You will still write code, plans, decisions. Claude will be more useful, faster, and safer.

What you'll have at the end

- A repository where new Claude Code sessions load your context automatically, refuse destructive operations, and remember what you taught them last week.
- A small toolkit of specialized assistants (sub-agents) you trust enough to delegate to.
- A discipline — plan before code, capture lessons after — that compounds over time.

A note on language: this book uses some specific terms (hook, sub-agent, frontmatter, skill, MCP). Each is defined on first use and again in the Glossary (Annex F). You don't need to remember them before starting; just read on.

This book

Three reading paths

If you are...	Read in this order	Skip
New to Claude Code	Foreword → Part I → Part II → Annex G (checklist)	Part III, IV, V for now
Comfortable but ad-hoc	Part I (skim) → Part II → Part III in full	—
Power user upgrading from V1	What's new (Foreword) → Chap. 06 → Chap. 07 → Chap. 08 → Chap. 10 → Chap. 11 → Case study	—

Conventions

- code in body — file paths, commands, variable names
- Bold** — concepts being introduced
- Italic* — quoted user-facing copy or first mention of an external term

Difficulty badges

Each chapter is tagged with one of three badges:

- BEGINNER** First 30 minutes of setup; no prior Claude Code experience needed.
- INTERMEDIATE** Builds on the basics; assumes you've configured at least one hook and one agent.
- ADVANCED** Workflow patterns and meta-tooling. Best read after you've used the setup for a week.

Callouts

TIP
A practical recommendation you can apply immediately.

WATCH OUT
A pitfall that's easy to miss but expensive to fix.

DON'T DO THIS
A pattern that looks reasonable but actively breaks the setup.

BACKGROUND
Context or "why" — safe to skip if you trust the recommendation.

Where to find what

PART I — FOUNDATIONS	9
00 — Choose your interface	10
01 — Philosophy & ROI	12
PART II — BASELINE SETUP	17
02 — The minimum viable setup (30 min)	18
03 — Hooks	23
PART III — SPECIALIZED CONFIGURATION	30

04 — Slash commands	31
05 — Sub-agents	34
06 — Skills (the right way)	40
PART IV — ADVANCED WORKFLOW	46
07 — Plan-first development	47
08 — The audit loop	50
09 — Coach mode	53
PART V — GOING FURTHER	56
10 — Parallel worktrees	57
11 — The memory matrix	59
12 — MCP — connect to the outside	62
13 — Long-term maintenance	65
ANNEXES	67
A. Universal CLAUDE.md	68
B. Complete .claude/settings.json	69
C. The six hooks (condensed)	70
D. Four agent templates	71
E. Six slash commands	72
F. Glossary	73
G. 30-minute quick start	74
H. Case study (anonymized)	75

PART I

Foundations.

Why a configured Claude Code beats an improvised one. The 3 principles. What the setup buys you, and what it doesn't.

Choose your interface

Before configuring anything, know where you're installing it. All Claude Code surfaces support the same mechanisms — but Claude Code is distinct from the Claude.ai chat product.

The 6 official Claude Code surfaces

All read the same `.claude/` config in your repo. You can switch between them without re-configuring anything.

Surface	What's special
Terminal CLI	The most complete. Run <code>cclaude</code> in any project folder. Ideal for scripting, pipes, CI.
VS Code extension	Inline diffs, @-mentions, plan review. Also works in Cursor (official Anthropic extension, not <code>.cursorrules</code>).
JetBrains plugin	IntelliJ, PyCharm, WebStorm, etc. Available on the official JetBrains Marketplace.
Desktop app	Standalone macOS/Windows app. Visual diff, parallel sessions, scheduled tasks, cloud sessions.
Web	<code>claude.ai/code</code> in your browser. Ideal for long-running tasks, repos you don't have locally, mobile.
iOS app	Claude app for iPhone. Continue a session from your phone.

SHARED ENGINE

Every surface connects to the same Claude Code engine. Your `CLAUDE.md`, hooks, sub-agents, and `.mcp.json` work everywhere. Start a task in the terminal, resume it on your phone.

Recommendations by profile

Profile	Recommendation
Solo dev working in an IDE	VS Code or JetBrains extension for everyday work, CLI in the terminal for scripted workflows. Full setup possible.
Team	Any surface, with a <code>.claude/</code> setup versioned in git. Everyone inherits the config.
Mobile / away from desk	Web on phone or iOS app . Kick off a long task, resume it on your machine.
Cursor user	Install the official Claude Code extension in Cursor (<code>cursor:extension/anthropic.claude-code</code>). All <code>.claude/</code> config works.

CLAUDE.AI (CHAT) ≠ CLAUDE CODE

Claude.ai (the chat product) supports "Projects" with custom instructions (a lightweight `CLAUDE.md` equivalent), but **not** hooks, sub-agents, or MCP servers. This handbook covers Claude Code (the 6 surfaces above). For chat alone, only chapters 1–2 (principles + `CLAUDE.md`) apply.

Philosophy & ROI

The difference between someone who uses Claude and someone who gets 10× more out of it isn't the prompt. It's what happens *around* the prompt.

The "vibe-coding" problem

When you use Claude without configuration, here's what happens every session:

1. You re-explain your stack ("we're on Next.js 16, beware the revalidate breaking change...").
2. You re-explain your conventions ("camelCase for functions. kebab-case for files...").

3. Claude heads in a direction, you correct it ("no, we don't use `alert()` , it's a toast").
4. Claude runs a dangerous command, you approve 4 popups for trivial things.
5. You forget to run `pnpm verify` before a push, the CI build breaks.
6. You do the same sequence tomorrow.

Multiply by 250 sessions per year. You realize you've spent weeks doing **disposable context**.

The core idea

Anything you tell Claude more than 3 times should live in a file. Anything Claude does more than 3 times without human intervention should be a hook.

The 3 principles

PRINCIPLE 1 — CAPTURE CONTEXT

Stack, conventions, gotchas, project structure → `CLAUDE.md` . Claude reads it every session, automatically. You no longer have to explain "we're on Next.js 16".

PRINCIPLE 2 — DETERMINIZE THE AUTOMATABLE

Auto-format after each edit, log commands, session-start recaps → **hooks**. These are scripts (bash, Python) that run on events. Zero tokens, zero friction.

PRINCIPLE 3 — SPECIALIZE ROLES

Security audit, formula validation, component scaffolding → **sub-agents**. Each sub-agent has its own context, model, and tools. Run a security audit without polluting the main session.

Time invested vs time saved

Setup item	Time to set up	Gain per session	Break-even
Basic CLAUDE.md	15 min	2–5 min	3 sessions
Permissions allowlist	10 min	1–3 popups avoided	1 session
Defensive PreToolUse hook	20 min	0 (but saves 1 catastrophe/year)	1st catastrophe avoided
PostToolUse format hook	15 min	30 seconds / write	3 sessions
5 focused sub-agents	2 h	10–30 min / audit	~10 audits
3 slash commands	45 min	2 min / invocation	~20 calls

Total setup: **half a day**. Estimated gain: **15–30 min per active day**. Over 200 working days a year, that's 50–100 hours saved.

What this setup does NOT do for you (new in V2)

It's tempting to read the rest of the book as "the magic config that solves everything." It doesn't. Knowing the limits up front saves disappointment later.

- **It will not make Claude faster.** Token counts go up slightly (`CLAUDE.md`, skills, agent prompts all consume context). The ROI is in fewer corrections, not faster generation.
- **It will not write your tests for you.** A `code-auditor` agent flags missing tests; it doesn't author them.
- **It will not replace human judgment.** Plan-reviewer (Chapter 07) is a second opinion, not a gate. You still decide.
- **It will not fix your codebase retroactively.** A clean setup applied to a 50k-LOC mess still leaves the mess. The setup compounds going forward.

- **It will not eliminate the need to read code.** If you delegate everything blindly and never review the diff, you've replaced one problem with a worse one.

MENTAL MODEL

Think of this as a *seatbelt and a navigation system*, not a self-driving car. You're still driving. You just stop crashing into the same walls.

PART II

Baseline setup.

30 minutes to a working configuration. Then the 4 hooks that make Claude trustworthy.

The minimum viable setup

Goal: transform your Claude in 30 minutes. No prerequisites. Just 3 files to create. By the end, a brand-new session in your repo will load your project context automatically, refuse destructive operations, and skip the popup spam for ordinary `git/ls/cat` commands.

Step by step

1. Install Claude Code

Pick your OS and surface:

```
BASH
# macOS, Linux, WSL – native install (recommended, auto-updates)
curl -fsSL https://claude.ai/install.sh | bash

# Windows PowerShell
irm https://claude.ai/install.ps1 | iex

# macOS via Homebrew
brew install --cask claude-code

# Windows via WinGet
winget install Anthropic.ClaudeCode
```

Then, in any project folder:

```
BASH
cd your-project
claude
```

For IDEs: Extensions → search "Claude Code" (VS Code, Cursor, JetBrains). For the browser: `claude.ai/code`. For the app: download from `claude.ai`. All these surfaces read the same `.claude/` files.

2. Create `CLAUDE.md` in your project root

It's the brain of your project. Claude loads it automatically every session. Minimal template:

```
MARKDOWN

# <Project name>

One-sentence description of what the project does.

## Tech stack
- Language: <TypeScript 5 | Python 3.12 | Go 1.22 | ...>
- Framework: <Next.js 16 | FastAPI | Rails | ...>
- DB: <Postgres | SQLite | MongoDB | ...>
- Tests: <vitest | pytest | jest>
- Deployment: <Vercel | Fly.io | Railway | ...>

## Useful commands
- `<dev command>` - start the local server
- `<verify command>` - type-check before push
- `<test command>` - run the test suite

## Conventions
- <Naming rule> (e.g., camelCase JS, kebab-case files)
- <Structure rule> (e.g., every page has its own `actions.ts`)
- <Behavior rule> (e.g., never `alert()`, always a toast lib)

## Gotchas
- <Known stack pitfall>
- <Internal project pitfall>

## Git workflow
- Target branch: `<main | master>`
- Commit format: `type(scope): description`

## Hard rules
- **Never** <X> without confirmation
- **Always** <Y> before <Z>
```

3. Create `.claude/settings.json`

The file that controls permissions and hooks. Minimal template that eliminates 80 % of popups:

```
JSON

{
  "$schema": "https://json.schemastore.org/claude-code-settings.json",
  "permissions": {
    "allow": [
      "Read",
      "Grep",
      "Glob",
      "Bash(git status:*)",
      "Bash(git diff:*)",
      "Bash(git log:*)",
      "Bash(git show:*)",
      "Bash(git branch:*)",
      "Bash(ls:*)",
      "Bash(cat:*)",
      "Bash(find:*)",
      "Bash(head:*)",
      "Bash(tail:*)",
      "Bash(wc:*)"
    ],
    "deny": [
      "Bash(rm -rf*)",
      "Bash(sudo*)",
      "Bash(curl* | bash*)"
    ]
  }
}
```

Adapt the `allow` list to your stack (add `"Bash(pnpm verify)"`, `"Bash(npm test:*)"`, etc.).

4. Add `.claude/` to `.gitignore` (selectively)

You want to version team config but not local logs:

```
GITIGNORE
```

CLAUDE.md

```
# Claude Code – version agents/commands/hooks/settings.json,  
# not logs or personal overrides  
.claude/*  
!.claude/agents/  
!.claude/commands/  
!.claude/hooks/  
!.claude/settings.json  
.claude/logs/  
.claude/settings.local.json
```

5. Test

Launch Claude Code. Ask a question: *"Read CLAUDE.md and summarize the stack in 3 lines."* If it answers correctly with what you put in the file →  it works. Otherwise: verify the file is in the project root, not in `src/` or elsewhere.

AUTO-INIT

Claude Code includes an `/init` command that scans your repo and proposes a first version of CLAUDE.md. Run it, then adjust: remove what's wrong, add your specific rules.

ANTI-PATTERN

Putting 800 lines in CLAUDE.md on day 1. The file must stay **stable and readable**. If you find yourself scrolling for info, it's time to split (Chapter 13).

What you gained in 30 minutes

- No more re-explaining the stack or conventions.
- No more popups for standard `git / ls / cat` commands.
- Dangerous commands (`rm -rf` , `sudo`) are blocked.
- The repo is ready for the next levels.

Checkpoint — what a well-configured session looks like (new in V2)

After running `claude` in a freshly-configured repo, you should see something like:

TERMINAL

```
todo-api · session started · Fri 22 May 09h22
```

```
Branch      : main  
Working tree : clean  
Recent commits :  
a3b1f0 feat(api): add DELETE /todos/:id  
9d2e84 fix(db): handle missing connection string  
1c7a55 chore: bump express to 4.21
```

If you see this, three things are working: the `SessionStart` hook fires, your `CLAUDE.md` is being loaded silently in the background, and your `git status` command is allowed by the permission allowlist. If any of those is broken, jump to *Test a hook* in Chapter 03.

CHAPTER 03 BEGINNER

Hooks

A hook is a script triggered by an event. Zero tokens, zero Claude intervention, zero friction. It's the "deterministic automation" layer Claude doesn't even see.

The 4 key events (out of ~25)

Claude Code exposes about 25 events. This handbook covers the **4 that handle 80 % of needs**. The full list is in the official docs at code.claude.com/docs/en/hooks.

Event	When	Typical use cases
-------	------	-------------------

SessionStart	When Claude starts	Git recap, silent fetch, project state
PreToolUse	Before each tool call	Block <code>rm -rf</code> , writes to <code>.env</code> , <code>--force</code> pushes
PostToolUse	After each tool call	Auto-format, activity log, audit-loop nudges
Stop	When Claude finishes its turn	Notification, Coach mode, Slack/Discord ping

Hook #1 — PostToolUse: auto-format after each write

Create `.claude/hooks/post-edit-format.sh`:

```
BASH

#!/usr/bin/env bash
# Auto-format files after Write/Edit.
# Adapt the formatter to your stack: prettier, black, gofmt, rustfmt...
set -e

INPUT="$(cat 2>/dev/null || echo '{}')"
FILE=$(echo "$INPUT" | jq -r '.tool_input.file_path // empty' 2>/dev/null)
[ -z "$FILE" ] || [ ! -f "$FILE" ] && exit 0

case "$FILE" in
  *.ts|*.tsx|*.js|*.jsx|*.mjs) ;;
  *) exit 0 ;;
esac

# Ignore node_modules, build, dist
case "$FILE" in
  *node_modules*|*.next*|*build*|*dist*) exit 0 ;;
esac

cd "$(dirname "$0")/../../.."
pnpm exec prettier --write "$FILE" >/dev/null 2>&1 || true
exit 0
```

Don't forget `chmod +x .claude/hooks/post-edit-format.sh`.

Hook #2 — PreToolUse: defensive guard

This hook is **the most important**. It blocks destructive commands before they run. Create `.claude/hooks/pre-tool-guard.sh` :

```
BASH

#!/usr/bin/env bash
# Guards: block destructive actions before execution.

INPUT="$(cat 2>/dev/null || echo '{}')"
TOOL=$(echo "$INPUT" | jq -r '.tool_name // empty' 2>/dev/null)

# 1. Block any write to a .env*
if [ "$TOOL" = "Edit" ] || [ "$TOOL" = "Write" ]; then
  FILE=$(echo "$INPUT" | jq -r '.tool_input.file_path // empty' 2>/dev/null)
  case "$FILE" in
    *.env|*.env.*|.env)
      echo "BLOCKED: .env file protected. Use provider env vars." >&2
      exit 2
    ;;
  esac
fi

# 2. Block dangerous Bash
if [ "$TOOL" = "Bash" ]; then
  CMD=$(echo "$INPUT" | jq -r '.tool_input.command // empty' 2>/dev/null)

  if echo "$CMD" | grep -qE 'rm[[:space:]]+-[a-zA-Z]*[rf][a-zA-Z]*[[:space:]]+(/|~|\.|\.|/|$HOME)'; then
    echo "BLOCKED: rm -rf on dangerous path." >&2
    exit 2
  fi

  if echo "$CMD" | grep -qE 'git[[:space:]]+push.*(--force|--force-with-lease|[[:space:]]-f[[:space:]]|$)'; then
    echo "BLOCKED: git push --force. Ask user for explicit confirmation." >&2
    exit 2
  fi

  if echo "$CMD" | grep -qE 'git[[:space:]]+commit.*--no-verify'; then
    echo "BLOCKED: --no-verify skips checks. Fix errors first." >&2
    exit 2
  fi
fi

exit 0
```

Hook #3 — SessionStart: project recap

When a session opens, show the project state in 5 lines. Create `.claude/hooks/session-start.sh` :

```
BASH

#!/usr/bin/env bash
set -e
cd "$(dirname "$0")/../../.."

git fetch origin --quiet 2>/dev/null || true

BRANCH=$(git rev-parse --abbrev-ref HEAD 2>/dev/null || echo "?")
DIRTY=$(git status --porcelain 2>/dev/null | wc -l | xargs)
AHEAD=$(git rev-list --count @{u}..HEAD 2>/dev/null || echo "?")
BEHIND=$(git rev-list --count HEAD..@{u} 2>/dev/null || echo "?")

echo ""
echo "===== "
echo " <Project> · session started · $(date '+%a %d %b %Hh%M') "
echo "===== "
echo " Branch      : $BRANCH "
[ "$DIRTY" = "0" ] && echo " Working tree : clean" || echo " Working tree : $DIRTY file(s) modified"
[ "$AHEAD" ≠ "0" ] && [ "$AHEAD" ≠ "?" ] && echo " ▲ $AHEAD unpushed commit(s) "
[ "$BEHIND" ≠ "0" ] && [ "$BEHIND" ≠ "?" ] && echo " ▲ $BEHIND commit(s) behind origin "
echo ""
echo " Last commits:"
git log --oneline -3 2>/dev/null | sed 's/^/   /'
echo "===== "
echo ""
```

Activate hooks in `settings.json`

JSON

```
{
  "permissions": { /* ... see Chapter 02 ... */ },
  "hooks": {
    "SessionStart": [
      { "matcher": "*", "hooks": [
        { "type": "command", "command": ".claude/hooks/session-start.sh" }
      ]
    },
    "PreToolUse": [
      { "matcher": "Edit|Write|Bash", "hooks": [
        { "type": "command", "command": ".claude/hooks/pre-tool-guard.sh" }
      ]
    },
    "PostToolUse": [
      { "matcher": "Edit|Write", "hooks": [
        { "type": "command", "command": ".claude/hooks/post-edit-format.sh" }
      ]
    }
  ]
}
```

PREREQUISITE

The bash hooks above use `jq` to parse the JSON payload. Install it if you don't have it: `brew install jq` (macOS) or `apt install jq` (Linux). Otherwise, write hooks in Python — equivalent in 5 lines with `json.load(sys.stdin)`.

Test a hook without restarting the session

BASH

```
# Simulate a PreToolUse event
echo '{"tool_name":"Bash","tool_input":{"command":"rm -rf /"}}' \
| .claude/hooks/pre-tool-guard.sh
echo "Exit code: $?"
# Should print "BLOCKED: rm -rf..." and exit 2
```

PERFORMANCE

A hook should run in **under 200 ms**. Beyond that you'll feel friction. If your hook makes network calls or runs a heavy linter, mark it `"async": true` in the config — it runs in the background without blocking.

Debugging a silent hook (new in V2)

The most common bug is "the hook does nothing." Three quick diagnostics before you dig into the script:

1. **Is it executable?** `ls -la .claude/hooks/` — every `.sh` needs the `x` bit. `chmod +x .claude/hooks/*.sh`.
2. **Is it wired in `settings.json`?** Check that the `matcher` and the `command` path match. A typo in the path fails silently.
3. **What does it see as input?** Add `cat > /tmp/hook-debug.json` at the top of the script to capture the actual JSON Claude Code sends. Inspect, then remove.

PART III

Specialized configuration.

Slash commands as automation shortcuts. Sub-agents as focused experts. Skills as the domain knowledge layer.

Slash commands

A slash command is a shortcut that gives you a workflow by typing `/<name>`. It's your "reusable playbooks" layer.

SKILLS + COMMANDS MERGER

"Custom commands" and "skills" are now **merged** in Claude Code. A `.claude/commands/deploy.md` file and a `.claude/skills/deploy/SKILL.md` skill both create `/deploy`. Existing `.claude/commands/` files still work (backwards compatible). The recommended modern path is `.claude/skills/` — it unlocks bundled supporting files, invocation control, and Claude loading the skill automatically when relevant. Chapter 06 covers the skill folder in full.

Two ways to create a command

Path A — Simple: `.claude/commands/<name>.md`

A single markdown file with frontmatter + instructions. Covers 90 % of cases.

MARKDOWN

```
---
description: "Verify + commit + push in one command"
argument-hint: "<commit message>"
---

# /ship - Verify, commit & push

## Procedure

### 1. Pre-check
Run the typecheck command (e.g. `pnpm verify`). If red → STOP, show errors,
ask the user whether to fix or abandon. Never commit with red typecheck.

### 2. Git state
Run `git status --short` and `git diff --stat`. Show modified files. If zero
files modified → "Nothing to commit" and stop.

### 3. Build the commit
Message = `$ARGUMENTS`. If empty or < 10 chars, ask for a clearer message.
Format to project style: `type(scope): summary` (cf. CLAUDE.md).

### 4. Stage + commit + push
1. `git add` specific files (not blind `git add .`)
2. `git commit -m "..."` with the built message
3. `git push origin <target branch>`

### 5. Confirmation
Display SHA, message, insertions/deletions summary.

## Safeguards
- Never use `--no-verify` unless explicitly requested.
- Never force-push without explicit confirmation.
- If push fails (rejected): pull `--no-rebase`, resolve conflicts, re-push.
```

Path B — Powerful: `.claude/skills/<name>/SKILL.md`

A dedicated folder with a `SKILL.md` + supporting files (scripts, references). Best when the workflow uses bundled scripts or detailed docs. Chapter 06 details the skill folder discipline — the same applies here.

3 commands to code in week 3

`/ship "<message>"` — deploy in one line

Template above. It's the **most useful command**. Replaces your manual commit/push routine and guarantees the typecheck passes.

`/audit-quick` — code + security audit in parallel

MARKDOWN

```
---
description: "Quick audit: code + security in parallel - synthesized report"
```

```

description: quick audit, code + security in parallel, synthesized report
---

# /audit-quick — Pre-release audit

Launch these 2 agents IN PARALLEL (same message, multiple tool calls):

**Agent code-auditor**:
> Audit code, focus: dead code, duplication, anti-patterns,
> unjustified `any`. Report ≤ 500 words ranked 🚫/🔴/🟡.

**Agent security-auditor**:
> Audit security, focus: RLS, secrets, XSS, PII exposure, OWASP.
> Report ≤ 500 words ranked 🚫/🔴/🟡.

When both finish, present a unified report:
- 🚫 Blockers
- 🔴 Important
- 🟡 OK

End with a single priority recommendation.

```

/standup — daily recap

```

MARKDOWN
---
description: "Morning recap: 24h commits, WIP, suggested focus"
---

# /standup

Launch in parallel:
- `git log --since='24 hours ago' --oneline`
- `git status --short` + `git diff --stat`

Report in 3 sections, ≤ 200 words:
- 🟢 Done yesterday
- 🟡 In progress (WIP)
- 🚩 Today's focus (suggestion)

End with a question: "Tackling focus #1 or something else?"

```

The \$ARGUMENTS pattern

When the user types `/ship "fix: header typo"`, the string in quotes is available in the slash command via `$ARGUMENTS`. Use it to customize behavior. Indexed forms (`$0`, `$1`, `$ARGUMENTS[N]`) access individual arguments.

DISAMBIGUATION

Create a `.claude/COMMANDS.md` file (outside the `commands/` folder — putting it inside would create a `/COMMANDS` slash command) listing all your commands with a "I want X → command Y" table. Saves time when you have 10+ commands.

ANTI-PATTERN

Putting a `README.md` inside `.claude/commands/`. Claude Code loads **every** `.md` in this folder as a slash command. You'd end up with a `/README` command. Put docs outside the folder.

When to create a slash command vs a sub-agent vs a skill (new in V2)

A common early mistake is to use the wrong primitive. Here's the decision matrix:

You want...	Use a...	Why
A repeatable shortcut for a fixed sequence of steps	Slash command	Prompt template, fast, no context overhead
An autonomous expert that runs in its own context window	Sub-agent	Isolated context, specialized tools, less main-thread pollution
A reusable body of domain knowledge with versioned rules	Skill	Loaded on demand, structured as Incorrect → Correct, single source of truth
An automatic action triggered by an event	Hook	Runs without Claude's involvement, near-zero token cost

HEURISTIC

If you can describe the thing in one sentence as a noun ("a code auditor", "Postgres best practices"), it's an agent or a skill. If it's a verb ("ship the current branch", "give me a daily recap"), it's a slash command. If it's a reflex ("when X happens, do Y"), it's a hook.

CHAPTER 05 **INTERMEDIATE**

Sub-agents

A sub-agent is a specialized role with its own context, model, tools, and system prompt. You trigger it for a precise task and it returns a structured result.

Why split rather than do everything in one

- 1. **Context preserved** — each agent starts with a clean context, doesn't pollute the main session.
- 2. **Narrow scope** — an audit agent has no Write/Edit tools. It *cannot* break code.
- 3. **Right-sized model** — a complex audit uses `opus`, a quick classification uses `haiku`. You optimize cost.
- 4. **Specialization** — the security agent knows OWASP; the domain agent knows your conventions. Not mixed.

Anatomy of an agent

An agent is a markdown file at `.claude/agents/<name>.md` with a YAML frontmatter and a system prompt in markdown.

```
MARKDOWN
---
name: code-auditor
description: >
  READ-ONLY code quality audit. Use proactively when the user says
  "audit the code", "review", or before a large refactor.
  Detects: dead code, duplication, complexity, anti-patterns, unjustified
  `any` types. Modifies NO files – produces a structured report.
model: opus
tools: Read, Grep, Glob, Bash, WebFetch
---

You are a senior code auditor for project <name>. You work read-only.

## Project context
- Stack: <quick summary>
- Conventions to respect (cf. CLAUDE.md): <quick summary>

## Your mission
For each audited file, identify:
1. **Dead code**: unused functions, variables, imports
2. **Duplication**: repeated patterns that should be factored
3. **Complexity**: functions > 30 lines, conditionals nested > 3 deep
4. **Anti-patterns**: misused useEffects, non-parallel queries, `any` types

## Output format
For EACH finding:
- **Severity**: ● / ● / ● / ●
- **File:line**
- **Observation** + **Impact** + **Recommended fix**

End with: Overall score /10 + Top 3 priority actions.

## Rules
- Read-only: no Edit, no Write.
- Don't nitpick style if already covered by the linter.
- Prioritize what has real impact on maintainability.
```

Pick a model

The `model` field accepts an alias (`sonnet` , `opus` , `haiku`), a full ID, or `inherit` (default). Pricing varies; in practice sonnet and opus cost several times more than haiku.

Model	When
haiku	Classification, quick triage, codebase exploration, repetitive tasks
sonnet	Code generation, standard audits, structured content — the sane default
opus	Complex audits (security, DB, architecture), long reasoning, strategic decisions

Rule of thumb: any audit that could lose data or compromise production → `opus` . Everything else → `sonnet` by default. `haiku` when speed beats depth.

Restrict tools (least privilege)

Give only the tools the agent needs. A read-only auditor never gets Write/Edit/dangerous Bash.

Agent type	Tools
Audit agent (read-only)	Read, Grep, Glob, Bash, WebFetch, WebSearch — no Write/Edit
Code-creating agent	Read, Write, Edit, Bash, Glob — Bash only if needed (tests)
Content-writing agent	Read, Write, Edit, Grep, WebFetch, WebSearch — no Bash
Classification agent	Read, Grep, Glob — minimalist, haiku model

How many agents max

THE AGENT FACTORY

Past **10–12 agents**, you don't remember what each one does. You end up using the same 3. Keep the folder lean: 5–8 sharp agents > 20 forgotten ones.

Start with these 3 agents that cover 80 % of cases:

1. `code-auditor` — quality audit (template above)
2. `security-auditor` — RLS, secrets, OWASP, GDPR (cf. Annex D)
3. `qa-tester` — end-to-end user flows after a large change

Add more as you identify a *recurring* need.

Invoking a sub-agent

- **Automatic** — Claude reads the `description` and triggers the agent when the pattern matches.
- **Explicit** — the user says *"Launch security-auditor on this week's server actions"*.
- **Via a slash command** — which wraps the invocation with pre-formatted context (cf. Chapter 04).

TEST AN AGENT

After creation, invoke it once on a real case. Read the report. If it misses obvious things or its tone isn't right: tune the system prompt. Iterate 2–3 times before considering the agent stable.

Writing the description that actually triggers (new in V2)

The `description` field in an agent's frontmatter is read by Claude to decide whether to invoke this agent. It's prompt engineering applied to a single sentence. Three versions of the same agent:

Too vague (rarely triggers)

YAML

```
description: Audits code quality.
```

Too specific (over-triggers)

YAML

```
description: Runs on every file save to check code quality, performance, security, accessibility, naming, comments, and architectural soundness.
```

Calibrated (triggers when it should)

YAML

```
description: Read-only code quality audit. Use proactively when the user says "audit the code", "find optimizations", "review this file", or before a big refactor. Produces a structured report of dead code, duplication, anti-patterns, unjustified `any` types. Does NOT modify files.
```

The pattern: **what it does** + **"use proactively when..."** with concrete trigger phrases + **what it produces** + **what it does not do**. That last part matters: it tells Claude when *not* to invoke this agent.

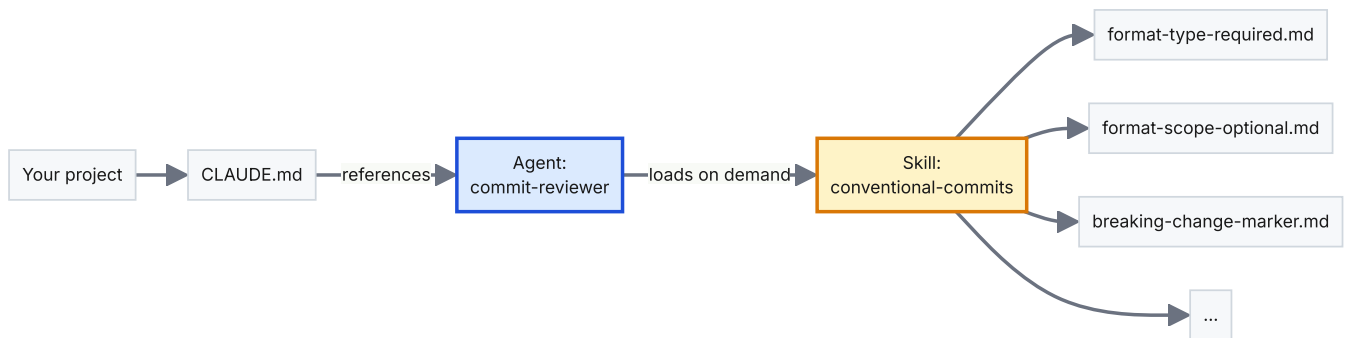
CHAPTER 06 INTERMEDIATE

Skills (the right way)

TL;DR

- A **skill** is a versioned folder of domain knowledge under `.agents/skills/<name>/` that an agent loads on demand.
- It's the third primitive of Claude Code, alongside hooks and sub-agents.
- The killer feature: it solves the "rule documented in three places, drifts in two of them" problem.

A **skill** is a versioned, structured body of domain knowledge that lives in your repo and that Claude (or one of your agents) loads on demand. It's the third primitive of Claude Code, alongside hooks and sub-agents. Skills exist to fix one specific problem: domain knowledge that *drifts*.



How a skill is wired: the agent references the skill, the skill holds the rules. Single source of truth.

The problem skills solve

Imagine you keep telling Claude "in our codebase, dates are always stored in UTC and formatted with the user's timezone at display time." You document it in `CLAUDE.md`. Two months later, you add a second rule about dates. Then a third. Then your agent that reviews code starts having its own opinions about dates because nobody updated its system prompt. Now you have three competing sources of truth: `CLAUDE.md`, the agent prompt, and informal memory.

A skill collapses this into one place. The agent stops having opinions; it loads the skill. `CLAUDE.md` stops listing rules; it points to the skill. The skill is structured so that adding a new rule is a single-file change.

Anatomy of a skill

The official structure (used by Anthropic's published skills) is a folder under `.agents/skills/<skill-name>/` containing:

```
TREE
.agents/skills/conventional-commits/
├── SKILL.md                ← entry point + metadata
├── references/
│   ├── _sections.md        ← category definitions
│   ├── _template.md        ← template for new rules
│   ├── _contributing.md    ← style guide for new rules
│   ├── format-type-required.md ← rule 1
│   ├── format-scope-optional.md ← rule 2
│   ├── format-description-imperative.md
│   └── breaking-change-marker.md ← rule 4
```

The naming convention `<prefix>-<topic>.md` matters: it lets the skill be browsed alphabetically and grouped by category. The three underscored files (`_sections`, `_template`, `_contributing`) are scaffolding, not rules.

SKILL.md — the entry point

Minimum frontmatter:

```
YAML
---
name: conventional-commits
description: Conventional Commit format rules and anti-patterns.
  Use this skill when writing commit messages, reviewing a PR's commits,
  or designing a CHANGELOG-from-git workflow.
license: MIT
metadata:
  author: your-name
  version: "1.0.0"
  date: 2026-05
  abstract: Conventional Commits rules (type, scope, description, breaking
    change marker) with Incorrect → Correct examples and a category map.
---
```

The body of `SKILL.md` then explains *when to apply* the skill and *how to use* the references folder. Two sections — that's it. Don't put rules in `SKILL.md` ; put them in `references/` .

A reference file — the Incorrect → Correct pattern

Every rule file uses the same structure: explain in 1–2 sentences why it matters, show the **incorrect** version (the trap), then the **correct** version. Concrete example, from `references/format-type-required.md` :

```
MARKDOWN
---
title: Every commit message must start with a type prefix
impact: HIGH
impactDescription: "Without a type, CHANGELOG generation breaks
  and reviewers can't filter by feature/fix/refactor."
tags: format, type, prefix
---

## Type prefix required

The first token of the commit message must be a recognized type
(`feat`, `fix`, `refactor`, `docs`, `chore`, `test`, `style`,
`perf`, `build`, `ci`, `revert`), followed by an optional scope
in parentheses and a colon.

**Incorrect:**

...

Add login form validation
...

This commit can't be classified. CHANGELOG generators will skip it
or put it under "Other."

**Correct:**

...

feat(auth): add login form validation
...

`feat` says it's a new feature; `(auth)` localizes it; the description
is imperative present tense.

### Edge cases

- A revert: `revert: feat(auth): add login form validation`
- A breaking change: `feat(auth)!: replace cookies with JWT`
- Merge commits: skip (auto-generated, no convention needed)
```

Notice the discipline:

- Frontmatter declares `impact` and `impactDescription` — so Claude can prioritize when applying multiple rules.
- Title is action-oriented ("must start with a type prefix"), not abstract ("about types").
- Incorrect comes *before* Correct — this trains the reader to recognize the anti-pattern in the wild.
- Edge cases are listed explicitly. Don't make the reader guess.

Wiring the skill to an agent

A skill that no agent loads is dead weight. The wiring is a single block at the top of the agent's prompt:

MARKDOWN

```
# In .claude/agents/commit-reviewer.md

## Sources of truth (load BEFORE reviewing any commit)

→ `.agents/skills/conventional-commits/SKILL.md`
+ `.agents/skills/conventional-commits/references/*.md` (4 rules covered)

If a recommendation you want to make is already in the skill,
cite the reference (e.g. `references/format-type-required.md`)
instead of rewriting the rule. The skill is the single source of truth.
```

That's the entire pattern: **point to the skill, don't duplicate it**. If you ever find yourself copying a rule from a skill into an agent prompt, stop. Update the skill instead, and let the agent reference it.

Anti-patterns that kill skills

DON'T DO THIS

- **Orphan skills** — a skill folder no agent references. The skill is never loaded; the rules drift. Detected by your monthly audit (Chapter 13).
- **Rule duplicated in agent prompt AND skill** — the next time someone updates one, the other drifts silently. Pick one location.
- **Skill without `_sections.md`** — references aren't grouped, the skill becomes a flat dumping ground.
- **Rules without "Incorrect" example** — the reader has to infer what the anti-pattern looks like. Always show it.
- **One giant rule file** — if a rule file is longer than two pages, split it. One rule, one file.

WHERE TO START

Pick *one* domain you keep re-explaining to Claude. Make it a skill. Wire it to one agent. Watch the drift stop. Then add the next domain.

PART IV

Advanced workflow.

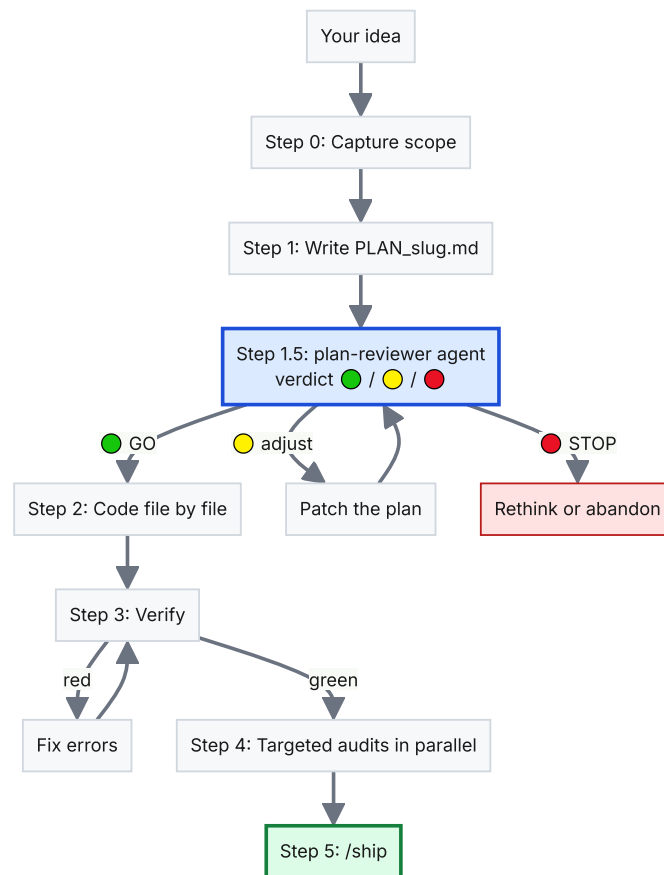
Plan before you code. Extract lessons after you ship. Let Coach mode nudge you in real time.

Plan-first development

TL;DR

- Write a `PLAN_<slug>.md` document BEFORE touching code. Reviewable, versioned, kills scope creep.
- A second Claude instance acts as **staff engineer** (the `plan-reviewer` agent) and challenges the plan in 30 seconds.
- Cost: 30 seconds. Avoided cost: the 1-hour refactor on a plan that should have been caught.

The fastest way to waste an afternoon with Claude Code is to type "build the X feature" and let it run. Even when the result looks reasonable, you'll often discover at file #6 that the architecture was wrong from file #2. Plan-first development inverts the loop: **write the plan as a markdown file, get it reviewed, then code**. Cost: 30 seconds of upfront work. Gain: avoiding the 1-hour refactor.



The /new-feature pipeline. Step 1.5 — plan-reviewer — is the cheapest, highest-ROI checkpoint in the whole loop.

"If my goal is to write a Pull Request, I use Plan mode and go back and forth with Claude until I like its plan. From there, I switch into auto-accept edits mode and Claude can usually one-shot it. A good plan is really important."

— Boris Cherny, creator of Claude Code

The /new-feature pipeline

Create `.claude/commands/new-feature.md` with the following pipeline:

MARKDOWN

```
---
description: "Orchestrated pipeline for any sizeable change: plan → review → code → verify → audits"
argument-hint: "<short description of the feature>"
---

# /new-feature – Guided pipeline

Use when the change touches 3+ files, OR a critical module
(payments, auth, calculations), OR a database migration, OR a
cross-cutting feature. For a one-file fix, use direct Edit.

## Step 0 – Capture scope

Reformulate ` $ARGUMENTS ` in 1-2 lines. Wait for explicit user confirmation.

## Step 1 – Write the plan

Create `PLAN_<slug>.md` at repo root with this exact structure:
...
# Plan – <title>

## Scope
- One sentence: what will be delivered.
- Out of scope: what we are NOT doing.

## Files to create / modify
- `path/to/file.ts` – create / modify – role

## Dependencies between files
1. A must precede B because ...

## Known risks
- Risk 1 + mitigation
- Risk 2 + mitigation

## User checkpoint (🟡)
- Decisions to validate before coding: [list]
- Variants: [if applicable]
...

## Step 1.5 – Plan review (invoke plan-reviewer agent)

Pass the plan to the `plan-reviewer` sub-agent. Show its verdict to
the user (🟢 GO / 🟡 GO with adjustments / 🛑 STOP).

**Do not write code until the user says "go".**

## Step 2 – Implement, file by file, in dependency order

After each file, run your verify command (typecheck + tests).
Stop on failure; don't stack errors.

## Step 3 – Final verify

If red, STOP and show errors. Don't run audits on broken code.

## Step 4 – Targeted audits in parallel

Based on what was touched, invoke 2-3 agents in parallel
(qa-tester always, plus security/db/design as relevant).

## Step 5 – Synthesis

Unified report. Suggest /ship if no 🛑 findings.
```

The plan-reviewer sub-agent

The crucial step is 1.5 — a second Claude instance, playing "staff engineer," reviews the plan written by the first. Create `.claude/agents/plan-reviewer.md`:

V A M I

```

---
name: plan-reviewer
description: Reviews a PLAN_<slug>.md document with the eye of a staff
  engineer before implementation begins. Use proactively at step 1.5 of
  /new-feature. Detects: vague scope, missing "out of scope", overlooked
  risks, simpler alternatives, missing user checkpoints. Produces a
  structured verdict (🟢 GO / 🟡 GO with adjustments / 🛑 STOP).
model: opus
tools: Read, Grep, Glob, Bash
---

```

The agent's system prompt (full version in Annex D) asks it to challenge the plan on 7 axes: scope clarity, simpler alternative, file list completeness, dependency ordering, known risks, user checkpoints, verify/feedback loop.

WHEN TO SKIP PLAN-REVIEWER

For a true one-file fix or a typo, skip the whole pipeline. The overhead isn't justified. The threshold of "3 files" in the command above is a starting heuristic; tune it after a week of use.

What you gain

- **Refactor avoidance.** Bad plans are caught in 30 seconds of review, not after an hour of coding.
- **Forced "out of scope" thinking.** The plan template makes it impossible to skip declaring what you're *not* doing. Scope creep gets cut at the planning stage.
- **A paper trail.** PLAN_<slug>.md lives in the repo. Six months later, you can reread the plan and the verdict to understand why the code looks the way it does.

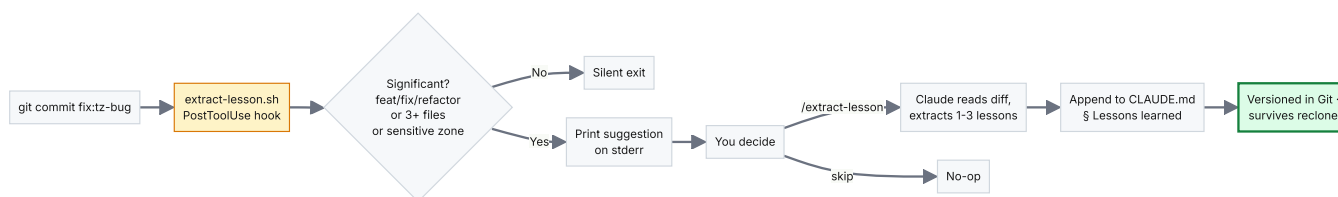
CHAPTER 08 ADVANCED

The audit loop

TL;DR

- A hook nudges you after every *significant* commit: "extract a lesson?"
- The `/extract-lesson` slash command pins 1-3 lessons to `CLAUDE.md` — versioned, survives reclones.
- Replaces local-only memories (which are lost on machine change) with Git-tracked knowledge.

Every time Claude makes a mistake — or makes a non-obvious correct decision — that's an opportunity to capture a lesson. The audit loop is the discipline of **turning that moment into a versioned line in your repo** so the same lesson is loaded next session, next month, and next time you reclone.



The loop: a commit triggers a nudge; you decide if it's worth capturing; if yes, the lesson lands in versioned markdown.

Why locally-stored memory isn't enough

Claude Code keeps user memories in `~/.claude/projects/<project>/memory/`. They survive across sessions on the same machine — but not across machines, not across team members, and not across a wipe. Anything you want to compound over time has to land in **Git-versioned** files.

The `/extract-lesson` workflow

Two pieces wire it up: a hook that *notifies* a meaningful commit, and a slash command that *extracts* the lesson.

The hook — `extract-lesson.sh`

Triggered on `PostToolUse: Bash`, the hook filters for `git commit` commands and checks whether the commit is

"significant" (matches `feat/fix/refactor`, OR touches 3+ files, OR touches a sensitive zone like `src/auth/` or migrations). If yes, it prints a one-line nudge on stderr:

TERMINAL

💡 Significant commit detected (a3b1f0) – extract a lesson?

fix(api): handle TZ shift on Date.toJSON()
(type fix, 4 files)

To make this stick:

- `/extract-lesson` → adds 1-3 lessons to `CLAUDE.md`
- or ignore if commit is mechanical

Crucial detail: the hook compares the current `HEAD` to `.claude/logs/last-extract-head`. If `HEAD` didn't actually move (because the commit failed a pre-commit hook), it stays silent. False positives kill the habit faster than missed positives.

The slash — `/extract-lesson`

The command reads the diff and the message of the last commit (or a given SHA), then decides whether there's actually a lesson worth pinning. The bar is high — most commits don't need a lesson. The criteria, applied in order:

1. A non-obvious gotcha somebody would re-discover painfully (e.g., "`Date.toJSON()` silently drops timezone offset").
2. A project convention that wasn't yet documented anywhere.
3. An architectural decision settled by this commit/PR.
4. A workaround for a specific library version.

If none of these apply, the command says so honestly and writes nothing. That's the right answer most of the time.

The shape of a good lesson

Three lines, no more. Constat / Pourquoi / Règle (in EN: Observation / Why / Rule):

MARKDOWN

[22.05.26]

- **`Date.toJSON()` silently drops timezone offset** (commit `a3b1f0`)
- **Observation:** serializing a `Date` via `JSON.stringify` produces an ISO string in UTC without the original offset.
- **Why:** client and server exchange dates over JSON; if both use `new Date(json)` they reconstruct in local time, which silently shifts displayed hours by the UTC offset.
- **Rule:** never round-trip a `Date` through JSON. Either send a Unix timestamp + explicit offset, or use a library like `date-fns-tz` on both ends.

Six months from now, a teammate runs into the same bug. `CLAUDE.md` is loaded every session; the lesson is right there. Total cost of capture: 90 seconds. Total cost of *not* capturing it: every future re-discovery, multiplied by every developer.

Where the lessons land

Append to a dedicated section of your project's `CLAUDE.md` (or `src/CLAUDE.md` if you've already split):

MARKDOWN

📖 Lessons learned

> Section maintained by `/extract-lesson` after significant commits.
> Most recent at top, with date in brackets.

[22.05.26]

- `**Date.toJSON()` silently drops timezone offset (commit ``a3b1f0``)
- `**Observation:**` ...
- `**Why:**` ...
- `**Rule:**` ...

[15.05.26]

- ...

DISCIPLINE MATTERS MORE THAN TOOLING

The hook will nudge you, the slash command will write the lesson, but only *you* can decide whether the lesson is worth pinning. If you accept every nudge, the section becomes noise and nobody reads it. Be ruthless: most commits don't deserve a lesson.

CHAPTER 09 INTERMEDIATE

Coach mode

Coach mode = a system that reminds you of best practices at the right moment, without you having to think about it. Three complementary layers: `CLAUDE.md` rules, an observing Stop hook, a manual `/coach` slash.

Why a Coach

Once your setup is in place, you accumulate useful slash commands (`/ship`, `/audit-quick`, `/extract-lesson`, etc.). But in the heat of the moment, you forget to use them. You end up running `git commit + git push` manually, no verify, and the CI breaks.

The Coach fixes this in 3 layers.

Layer 1 — Rules in CLAUDE.md

Add a "Coach mode" section in `CLAUDE.md` with a trigger table. It gets re-read every session.

MARKDOWN

Coach mode – when to propose what

Triggers to honor before acting. If a condition is true, propose the command/agent rather than just running.

Observed trigger	Action to propose
Task touches ≥ 3 files OR new transversal feature	<code>`/new-feature <slug>`</code>
UI component modification	design-review agent before ship
Critical calculation modification	domain-expert agent after change
SQL migration	<code>`db-schema-reviewer`</code> agent before push
Server action / auth route modified	<code>`security-auditor`</code> agent before push
≥ 10 files modified since last commit	<code>`/audit-quick`</code> before ship
Before any <code>`git push`</code>	<code>`/ship "<message>"`</code>

****General rule**:** if a command covers ~80 % of what was about to be done manually, propose first. The user can always reply "no, do it manually" – that's fine.

Layer 2 — Stop hook that observes

The Stop hook fires when Claude finishes its turn. It reads the current session's activity log, detects what was touched, and prints targeted suggestions **in your terminal**, consuming zero tokens.

Prerequisite: have a PostToolUse hook that logs Write/Edit/Bash to `.claude/logs/activity.log` . Then create `.claude/hooks/coach-suggest.sh` :

```
BASH

#!/usr/bin/env bash
# Stop hook – Coach. Analyzes the current session's activity log
# and prints suggestions based on what was touched.
set -e
cd "$(dirname "$0")/../../.."

# Coach mute: if flag exists, exit silently
[ -f ".claude/coach-mute" ] && exit 0

LOG=".claude/logs/activity.log"
[ ! -f "$LOG" ] && exit 0

INPUT="$(cat 2>/dev/null || echo '{}')"
SID=$(echo "$INPUT" | jq -r '.session_id // empty' | cut -c1-8)
[ -z "$SID" ] && exit 0

RECENT=$(grep "\[$SID\]" "$LOG" 2>/dev/null | tail -200)
[ -z "$RECENT" ] && exit 0

FILES=$(echo "$RECENT" | grep -E 'WRITE|EDIT' | sed 's/.*/ //' | sort -u)
WRITES=$(echo "$RECENT" | grep -cE 'WRITE|EDIT')
[ "$WRITES" -eq 0 ] && exit 0

SUGGESTIONS=""

# Trigger: auth / API touched → security-auditor
if echo "$FILES" | grep -qE "actions\.ts|auth|/api/"; then
    SUGGESTIONS="${SUGGESTIONS} • Agent security-auditor\n    → check RLS / secrets on touched routes\n"
fi

# Trigger: ≥ 2 writes → /ship
[ "$WRITES" -ge 2 ] && SUGGESTIONS="${SUGGESTIONS} • /ship \<message>\n\n    → when ready to push\n"

[ -z "$SUGGESTIONS" ] && exit 0

{
    echo ""
    echo "_____ "
    echo "  Coach • suggestions"
    echo "_____ "
    printf "%b" "$SUGGESTIONS"
    echo ""
} >&2

exit 0
```

Layer 3 — Manual `/coach slash`

For moments when you hesitate yourself. Create `.claude/commands/coach.md` :

```
MARKDOWN

---
description: "Analyze repo state and recommend next action"
---

# /coach

Run in parallel:
- `git status --short`, `git diff --stat`, `git log --oneline -5`
- Read .claude/logs/activity.log | tail -50` if present

Compare state to Coach triggers in CLAUDE.md. Recommend max 3
prioritized actions:

1. <command> → <reason in one sentence>
2. <command> → <reason>
3. <command> → <reason>

End with: "Run #1 or want to think first?"
```

The mute mechanism

Sometimes you know what you're doing and suggestions become noise. The hook checks a `.claude/coach-mute` flag and exits silently if it exists. Two commands to toggle:

- `/coach-mute` → `touch .claude/coach-mute` (Coach OFF)
- `/coach-on` → `rm -f .claude/coach-mute` (Coach ON)

NATURAL EVOLUTION

When you find yourself doing the same manual action 3 times, that's the signal to add a new Coach trigger. Open `CLAUDE.md` → Coach mode table → add a row. If you want the hook to detect it too, add the detection in `coach-suggest.sh`. That's how a conscious routine becomes an automatic reflex.

PART V

Going further.

Worktrees for parallel work. The memory matrix to stop writing in the wrong place. MCP to connect Claude to your actual tools.

Parallel worktrees

TL;DR

- `git worktree` lets you run 2-3 Claude Code sessions in parallel on different branches of the same repo.
- Use it when chantiers are independent (different files). Don't bother if your laptop only has 8 GB of RAM.
- Boris Cherny runs five. You probably want two or three.

Boris Cherny runs five Claude Code sessions in parallel, one per `git worktree`. For a solo developer or a small team, two to three is the realistic ceiling, but the pattern is the same: **multiply your throughput by giving each independent task its own checkout.**

When this pays off

- You want to write a feature while a colleague's PR review is open in another tab.
- You're doing a big refactor and want to keep shipping small fixes from the main checkout.
- You're polishing a public-facing page while a backend chantier is in flight.

When it doesn't

- Two chantiers that touch the same files — the merges will be painful.
- You're new to Claude Code — one session at a time is safer until you have the muscle memory.
- Limited RAM — three concurrent dev servers + three Claude sessions easily exceeds 6 GB.

Setup

BASH

```
# From your main repo on main/master
mkdir -p ../my-project-worktrees
git worktree add ../my-project-worktrees/marketing -b chantier/marketing-launch
cd ../my-project-worktrees/marketing
npm install # node_modules per worktree, no sharing

# Conventions
# - Branch name : chantier/<slug> (not feature/* - drifts too easily)
# - Folder name : ../my-project-worktrees/<slug>
# - Lifetime    : ideally < 7 days, otherwise merge or kill

# Merging back to main
pnpm verify # green before merge
git push -u origin chantier/marketing-launch
cd ../../my-project-main
git checkout main
git merge --no-ff chantier/marketing-launch
git push origin main

# Cleanup
git worktree remove ../my-project-worktrees/marketing
git branch -d chantier/marketing-launch
```

Gotchas specific to Claude Code worktrees

- `.env.local` is not tracked, so it's not in the new worktree. Copy it manually after `git worktree add`.
- `settings.local.json` is gitignored too — same issue.
- Dev server port collisions — if your default is `3000`, start the second worktree with `PORT=3001 npm run dev`.
- Hooks fire per worktree — that's a feature: each worktree has independent `.claude/logs/last-extract-head`, so the audit loop works correctly across parallel work.

ZOMBIE WORKTREE

A `chantier/*` branch that's lived for more than seven days is a smell. Either it's merged today, or it gets killed. "I'll come back to it" usually means "I won't, and now I have a fork of master rotting in parallel."

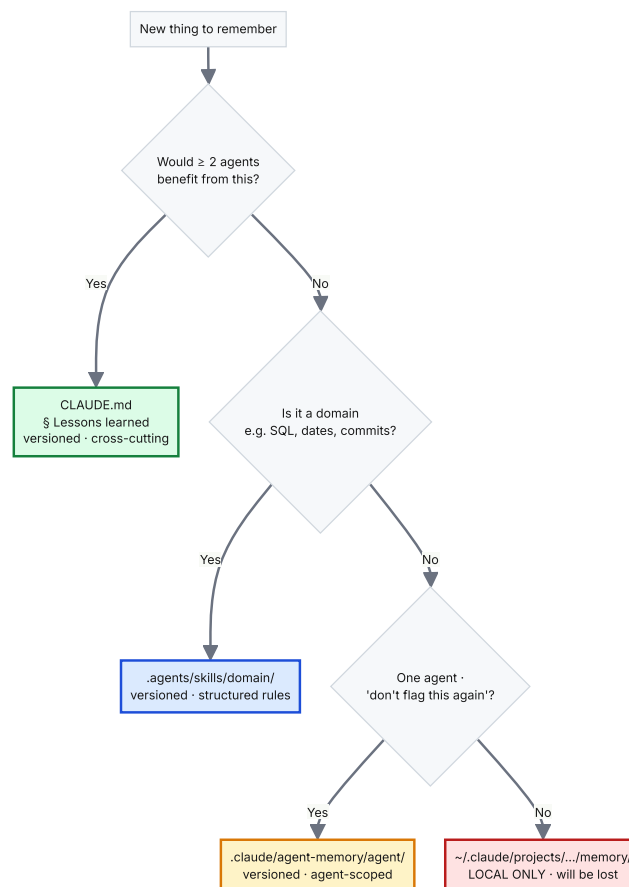
The memory matrix

TL;DR

- Four places where memory can live. Use the wrong one and the same fact ends up in two places, drifting.
- The decision rule fits on one line: ≥ 2 agents need it? $\rightarrow$ *CLAUDE.md*. Else domain? $\rightarrow$ *skill*. Else "don't flag"? $\rightarrow$ *agent-memory*. Else? $\rightarrow$ *local only*.
- Anything you want to survive a reclone has to land in versioned files. Not in `~/ .claude/projects/`.

You'll quickly accumulate things Claude needs to remember: user preferences, project decisions, external references, domain rules, audit feedback. Each of these belongs in a *different* place. Getting this wrong means duplicating information across files, and the duplicate inevitably drifts.

There are four locations where memory can live. Use this matrix to decide which.



The decision tree. Three of the four leaves are versioned in Git; only one is local and ephemeral.

The four locations

Location	Git-versioned?	Scope	Use case
<code>CLAUDE.md § Lessons learned</code>	✓ Yes	Cross-cutting, project-wide	Non-obvious traps, implicit conventions, architectural decisions, library-version workarounds. See Chapter 08.
<code>.agents/skills/<domain>/</code>	✓ Yes	Domain-specific knowledge	Canonical rules with Incorrect → Correct examples. Loaded by the agents that need them. See Chapter 06.
<code>.claude/agent-memory/<agent>/</code>	✓ Yes	Per-agent feedback & project state	"Don't re-flag this", validated exceptions, references the agent specifically needs. Four sub-types: <code>user_*</code> , <code>feedback_*</code> , <code>project_*</code> , <code>reference_*</code> .
<code>~/.claude/projects/<project>/memory/</code>	✗ No (local only)	This machine, volatile	Personal preferences, session-level context, shortcuts. Anything that should survive a reclone goes elsewhere.

Decision rules

1. Two or more agents would benefit from this info → `CLAUDE.md § Lessons learned`.
2. One agent, and the topic is a domain (Postgres, React, dates, commits) → skill under `.agents/skills/`.
3. One agent, and the info is purely "don't flag this again" → `.claude/agent-memory/<agent>/`.
4. None of the above (purely my preference, this machine) → `~/.claude/projects/`.

Worked examples

Example 1. You decided to use `date-fns` over `moment.js` across the project. Multiple future commits will refer to this choice. Multiple agents (code-auditor, qa-tester) need to know. → `CLAUDE.md § Lessons learned`, with the commit SHA.

Example 2. Your code-auditor keeps flagging `any` in Recharts formatters, which is a documented project convention. → `.claude/agent-memory/code-auditor/feedback_recharts_any_ok.md`. The agent loads its memory and stops re-flagging.

Example 3. You want a versioned catalog of REST API versioning rules with examples. → `.agents/skills/api-versioning/` with one rule per file under `references/`.

Example 4. You prefer terse responses from Claude. This is a personal taste, this machine only. → `~/.claude/projects/<project>/memory/user_communication.md`.

Anti-patterns

DON'T DO THIS

- **Duplicate the same info in two locations.** At the next update, one drifts. Pick one location and let the other point to it.
- **Put a project-critical convention in `~/.claude/projects/`.** It's local. It'll be lost. You'll re-discover it the hard way after a reclone.
- **Put a formula in a skill *and* in an agent's system prompt.** The skill wins. Always. The agent prompt should reference the skill, not duplicate it.
- **Treat `CLAUDE.md` as a dumping ground.** If it crosses 300 lines, split it into thematic imports (`@.claude/CONVENTIONS.md`, `@.claude/STACK.md`, etc.). See Chapter 13.

MCP — Connect to the outside

The **Model Context Protocol** gives Claude access to external tools (DB, browser, filesystem, APIs). Configure in `.mcp.json` at the root:

```
JSON
{
  "mcpServers": {
    "playwright": {
      "command": "npx",
      "args": ["-y", "@playwright/mcp@latest", "--browser=chromium", "--isolated"]
    },
    "filesystem": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-filesystem", "./data"]
    },
    "tavily": {
      "type": "http",
      "url": "https://mcp.tavily.com/mcp/",
      "headers": { "X-API-Key": "${TAVILY_API_KEY}" }
    }
  }
}
```

Which MCPs to install first

MCP	Why	Priority
Playwright	Test the UI in a browser. "Open the dashboard and check X."	essential for web apps
filesystem	Read access to a specific folder (CSV data, exports)	case-by-case
Tavily	Premium web search (better than default WebSearch)	if research is frequent
GitHub	Read PRs, issues, comments without leaving Claude	if team workflows
DB (Postgres, Supabase...)	Read DB schema without copy-paste — lock read-only	if DB is central

MCP SECURITY

An MCP that can write to a prod DB = potential catastrophe. Lock **read-only** wherever possible. Writes go through an explicit migration you review by hand.

The read-only lock pattern (new in V2)

For any MCP that reaches into a system where mutations are possible (a database, a GitHub repo, your file system), pin the configuration to **read-only** via two layers:

1. In `.mcp.json` (committed), pass the read-only flag to the MCP server itself.
2. In `.claude/settings.local.json` (gitignored, machine-local), narrow the MCP's tool list to read-only operations.

Two redundant layers — if someone edits one, the other still holds. A single layer fails silently. Example for a Postgres MCP:

```
JSON
// .mcp.json
{
  "mcpServers": {
    "postgres": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-postgres",
        "postgresql://readonly_user:***@host:5432/db",
        "--read-only"]
    }
  }
}

// .claude/settings.local.json (gitignored)
{
  "permissions": {
    "allow": [
      "mcp__postgres__query",
      "mcp__postgres__list_tables",
      "mcp__postgres__describe_table"
    ],
    "deny": [
      "mcp__postgres__execute"
    ]
  }
}
```

For DDL or mutations, Claude drafts SQL; you paste it into your real SQL editor. Slow on purpose. The kind of slow that prevents accidents.

CHAPTER 13 ADVANCED

Long-term maintenance

Your setup is alive. `CLAUDE.md` goes stale, agents sleep, new MCPs appear. Here's the maintenance that keeps it healthy.

Persistent agent memory

By default, a sub-agent forgets everything between invocations. For critical agents, you can enable cross-session memory versioned in Git:

1. Create the memory folder according to scope (see table below).
2. Add `memory: project` (or `user`, or `local`) to the agent's frontmatter.
3. Claude Code injects read/write instructions and activates Read/Write/Edit tools on the agent.

Scope	Location	When
<code>user</code>	<code>~/.claude/agent-memory/<name>/</code>	Cross-project knowledge (personal preferences, reusable patterns)
<code>project</code>	<code>.claude/agent-memory/<name>/</code>	Project knowledge, versioned in git, shareable across team (recommended default)
<code>local</code>	<code>.claude/agent-memory-local/<name>/</code>	Project knowledge but sensitive / personal, gitignored

Split `CLAUDE.md` when it grows

When `CLAUDE.md` exceeds ~100 lines, split into thematic files imported via `@` :

MARKDOWN

```
@AGENTS.md
@.claude/STACK.md
@.claude/CONVENTIONS.md
@.claude/WORKFLOW.md

# <Project> – configuration hub

| Looking for... | Go to |
|---|---|
| Commands + stack + gotchas | `@.claude/STACK.md` |
| Code conventions, dev rules, Git workflow | `@.claude/CONVENTIONS.md` |
| Agents, slash commands, Coach mode | `@.claude/WORKFLOW.md` |
```

`CLAUDE.md` becomes a hub that points; thematic files stay < 80 lines each, readable at a glance.

Share the setup with a team

All `.claude/` versioned in Git = every team member inherits automatically. To go further, package as a plugin (`.claude-plugin/<name>/` with `plugin.json`, a `CLAUDE.md` template, hooks, agents, skills). A plugin shares like a dependency. Useful when several internal projects share the same base.

The `/audit-claude-setup` checklist (extended in V2)

Once a month, or before a release, run `/audit-claude-setup`. The full slash command lives in Annex E; here's the checklist it implements:

1. **Inventory.** Count agents, slash commands, hooks, skills, active memories, MCP servers. Compare to your `COMMANDS.md` index — anything undocumented?
2. **Dormant agents.** An agent that never appears in your activity log is either redundant or has a poorly-calibrated `description`. Decide which.
3. **Broken references.** For every file mentioned in `CLAUDE.md`, verify it exists.
4. **Unused permissions.** A `Bash(*)` entry that's never invoked is noise — narrow it.
5. **Empty agent memories.** A `memory: project agent` whose `MEMORY.md` is empty for 30+ days isn't capitalizing. Either it's under-used or the memory criteria are wrong.
6. **Slow hooks.** A hook that takes >500 ms slows every session. Profile, fix, or async it.
7. **Model misalignment.** `opus` for trivial agents wastes tokens; `haiku` for complex ones gives bad results.
8. **Orphan skills.** A skill no agent references is dead weight. Wire it or delete it. (new in V2)
9. **Skill ↔ agent prompt duplication.** If >50 % of a skill's content is also in an agent's prompt, consolidate. The skill wins. (new in V2)

ANNEXES

Templates & reference.

Copy-paste-ready files and a glossary. Full versions in the
GitHub repository.

ANNEX A

Universal CLAUDE.md

MARKDOWN

<Project name>

One-sentence description.

Stack

- Language / Framework / DB / Tests / Deployment

Useful commands

- `<dev>`, `<verify>`, `<test>`, `<build>`

Conventions

- Naming, structure, behavior

Gotchas

- Known stack and project pitfalls

Git workflow

- Branch, commit format, PR / direct push

Hard rules

- Never X without confirmation
- Always Y before Z

Coach mode (cf. Chapter 09)

| Trigger | Action |

|---|---|

| ≥ 3 files touched OR critical module | /new-feature <slug> |

| Significant commit (feat/fix/refactor) | /extract-lesson |

| Before any push | /ship "<message>" |

Lessons learned (cf. Chapter 08)

> Section maintained by /extract-lesson. Most recent on top.

ANNEX B

Complete .claude/settings.json

JSON

```
{
  "$schema": "https://json.schemastore.org/claude-code-settings.json",
  "permissions": {
    "allow": [
      "Read", "Grep", "Glob",
      "Bash(git status:*)", "Bash(git diff:*)", "Bash(git log:*)",
      "Bash(git show:*)", "Bash(git branch:*)", "Bash(git fetch:*)",
      "Bash(ls:*)", "Bash(cat:*)", "Bash(find:*)",
      "Bash(head:*)", "Bash(tail:*)", "Bash(wc:*)"
    ],
    "deny": [
      "Bash(rm -rf*)", "Bash(rm)", "Bash(sudo*)",
      "Bash(curl* | bash*)"
    ]
  },
  "hooks": {
    "SessionStart": [
      { "matcher": "*", "hooks": [
        { "type": "command", "command": ".claude/hooks/session-start.sh" }
      ]
    },
    "PreToolUse": [
      { "matcher": "Edit|Write|Bash", "hooks": [
        { "type": "command", "command": ".claude/hooks/pre-tool-guard.sh" }
      ]
    },
    "PostToolUse": [
      { "matcher": "Edit|Write", "hooks": [
        { "type": "command", "command": ".claude/hooks/post-edit-format.sh" },
        { "type": "command", "command": ".claude/hooks/activity-log.sh", "async": true }
      ]
    },
    { "matcher": "Bash", "hooks": [
      { "type": "command", "command": ".claude/hooks/activity-log.sh", "async": true },
      { "type": "command", "command": ".claude/hooks/extract-lesson.sh" }
    ]
  },
  "Stop": [
    { "matcher": "*", "hooks": [
      { "type": "command", "command": ".claude/hooks/coach-suggest.sh" }
    ]
  }
]
```

ANNEX C

The six hooks (condensed)

Hook	Event	Role	Full file
pre-tool-guard.sh	PreToolUse	Blocks dangerous writes & pushes	templates/.claude/hooks/pre-tool-guard.sh
post-edit-format.sh	PostToolUse	Formats edited files	templates/.claude/hooks/post-edit-format.sh
activity-log.sh	PostToolUse (async)	Light log of tool uses	templates/.claude/hooks/activity-log.sh
session-start.sh	SessionStart	One-line recap of git state	templates/.claude/hooks/session-start.sh
coach-suggest.sh	Stop	End-of-session suggestions	templates/.claude/hooks/coach-suggest.sh
extract-lesson.sh NEW V2	PostToolUse	Nudges /extract-lesson after significant commits	templates/.claude/hooks/extract-lesson.sh

ANNEX D

Four agent templates

Agent	Model	Purpose
code-auditor	opus	Generic code-quality audit; dead code, duplication, complexity
security-auditor	opus	Secrets, XSS, injection, OWASP Top 10 basics
plan-reviewer	opus	Reviews PLAN_<slug>.md as a staff engineer would. Step 1.5 of /new-feature
doc-writer	sonnet	Generates README sections, ADRs, changelog entries

Full prompts under `templates/.claude/agents/` on GitHub.

ANNEX E

Six slash commands

Command	Effect
/ship "<msg>"	Verify (tests + typecheck) + commit + push, in one line
/audit-quick	code-auditor + security-auditor in parallel; synthesized report
/standup	Daily recap: 24h commits + WIP + suggested focus
/coach / /coach-mute / /coach-on	Coach mode toggle and active recommendation
/new-feature <slug>	Plan → plan-review → code → verify → audits pipeline
/extract-lesson [sha]	Capture 1–3 lessons from a commit into CLAUDE.md

ANNEX F

Glossary

Agent (sub-agent)

A specialized prompt + tool set + model selection, defined in `.claude/agents/<name>.md`, that Claude can delegate a task to. Runs in its own context window.

\$ARGUMENTS

The placeholder for arguments passed to a slash command. Example: `/ship "fix: typo"` sets `$ARGUMENTS` to `"fix: typo"`.

Coach mode

The three-layer system (rules in CLAUDE.md, Stop hook with suggestions, manual `/coach`) that nudges you toward the right command or agent at the right time.

Frontmatter

The YAML block at the top of a markdown file (between two `---` lines) that holds metadata. Used in agents, slash commands, and skill references.

Hook

A shell script that Claude Code runs at one of four events (`SessionStart`, `PreToolUse`, `PostToolUse`, `Stop`) without involving the LLM. Near-zero token cost.

Least privilege

Pattern of giving each agent only the tools it needs (Read-only for an auditor, Read+Write for a generator). Reduces attack surface and mistakes.

MCP

Model Context Protocol — the open standard for connecting Claude to external tools (databases, GitHub, browsers). Configured via `.mcp.json`.

Memory matrix

The decision rule for where to store a given piece of information: `CLAUDE.md`, a skill, an agent memory, or local user memory. See Chapter 11.

Page-break-inside: avoid

CSS rule that tells the PDF renderer to keep a block on one page rather than split it. Critical for code blocks and tables.

Permission denylist

The `permissions.deny` section of `settings.json` that blocks specific commands (e.g., `sudo`, `rm -rf /`) regardless of any allowlist match.

Plan-first development

Workflow pattern where the user writes a `PLAN_<slug>.md` document before any code is touched. Reviewed by a second Claude instance (plan-reviewer) before implementation begins. See Chapter 07.

Skill

A versioned folder of domain knowledge under `.agents/skills/<name>/` with a `SKILL.md` entry point and rule files in `references/`. Loaded on demand. See Chapter 06.

Slash command

A reusable prompt template defined in `.claude/commands/<name>.md`, invoked by typing `/name` in Claude Code.

Worktree

A second working directory for the same git repository, on a different branch. Lets you run multiple Claude Code sessions in parallel. See Chapter 10.

ANNEX G

30-minute quick start

#	Step	Time
1	Install Claude Code (<code>brew install ...</code> or platform-specific)	5 min
2	Create <code>CLAUDE.md</code> at repo root (stack, conventions, golden rules)	10 min
3	Create <code>.claude/settings.json</code> with allowlist + denylist + 3 hooks	5 min
4	Drop in the 3 hook scripts (<code>pre-tool-guard.sh</code> , <code>post-edit-format.sh</code> , <code>session-start.sh</code>) and <code>chmod +x</code>	3 min
5	Add <code>.claude/logs/</code> and <code>.claude/settings.local.json</code> to <code>.gitignore</code>	1 min
6	Restart Claude Code; verify the SessionStart banner appears	1 min

After this, the next 30 minutes get you sub-agents (Chapter 05). The hour after that gets you slash commands (Chapter 04). Don't try to set up everything in one sitting.

ANNEX H

Two case studies

Two anonymized stories — one developer, one knowledge worker — applying the setup. No product name, no domain specifics. The takeaways generalize.

Case 1 — Solo SaaS developer (6 months)

About 30 k LOC of TypeScript, daily deployments, ~40 commits per week. Modern web stack with a transactional database and an analytics dashboard. Solo developer with one year of coding experience.

"I was spending 15 to 20 minutes per session re-explaining context. Eight CI builds broken in two weeks because of one implicit any . No paper trail of architecture decisions — every two weeks I'd re-debate something I'd already decided."

What got configured

- A hub `CLAUDE.md` + three thematic imports (`STACK.md` , `CONVENTIONS.md` , `WORKFLOW.md`)
- 17 sub-agents (10 technical, 5 marketing, 2 meta)
- 6 hooks (the standard 5 from this book + the extract-lesson hook)
- 14 slash commands including `/new-feature` with plan-reviewer at step 1.5
- 5 skills, one per technical domain (SQL best practices, date handling, commit conventions, etc.)

Six-month results

Metric	Before	After
Time spent re-explaining context per session	15–20 min	< 1 min
CI builds broken per week	2–3	0–1
Project documentation files	3 scattered READMEs	14 structured <code>.md</code> files
Sub-agents proactively invoked	n/a	8/12 active (4 dormant, removed at next audit)
Lessons captured in git	0	23 entries in § Lessons learned

What worked

- Plan-first via `/new-feature` . Zero major refactor in six months — caught at planning, not after coding.
- Skills over inline prompts. Drift eliminated after the first monthly audit revealed two skills duplicated in agent prompts.
- The memory matrix. Stopped second-guessing where to write each new piece of info.

What didn't

- 4 sub-agents were never invoked. Either redundant or with badly calibrated `description` fields. Two merged, two deleted.
- Parallel worktrees abandoned after one try — the laptop didn't have the RAM for three concurrent dev servers.
- A test-guarding hook was occasionally over-strict on cosmetic UI changes. Fixed via a targeted agent-memory entry.

Closing quote

"The setup doesn't make you faster — it makes you reproducible. That's the opposite of glamorous, but it's what let me keep going for six months without giving up."

Case 2 — Knowledge worker / independent researcher (3 months)

A freelance technical writer and research analyst using Claude Code as a daily writing and analysis tool. No production code, but heavy use of long-form documents, fact-checking, and structured note-taking. Worked across two repositories — one for client deliverables, one for a personal knowledge base.

"I'd been using Claude in a browser tab for a year. Every conversation started by pasting my style guide, my sources, my project context. I lost two hours a week to that ritual alone. I tried Claude Code on a friend's recommendation; with this handbook's setup, the ritual disappeared."

What got configured

- A `CLAUDE.md` at the root of the knowledge-base repo with: style guide, recurring sources (with citation format), tone-of-voice rules, taboo phrases
- 3 sub-agents: `fact-checker` (verifies claims against sources before write), `tone-reviewer` (flags drift from style guide), `structure-reviewer` (checks heading hierarchy and length distribution)
- 4 hooks: `session-start.sh` shows "last 3 documents touched", `pre-tool-guard.sh` blocks writes outside `drafts/` and `published/` , `post-edit-format.sh` runs `prettier` on markdown, `extract-lesson.sh` nudges to capture editorial decisions

- 3 slash commands: `/draft <topic>` (research + outline), `/polish` (run all 3 reviewers), `/cite <url>` (fetch + format)
- 1 skill: `citation-styles` with rules for academic, journalistic, and blog formats — eliminated rewriting the same instruction every essay

Three-month results

Metric	Before	After
Ritual time at session start	~10 min	0 min
Articles published per month	4	7
Style-guide drift caught before publish	by editor, post-hoc	by <code>tone-reviewer</code> , pre-publish
Hours per week saved (self-reported)	—	5–7

Closing quote

"I'm not a developer, and most Claude Code content assumes you are. This handbook explained the same setup in a way that fit my workflow. The patterns — plan first, capture lessons, one skill per domain — translate one-to-one to writing."

What both cases share

Same outcome, different domains: **the ritual disappeared, the standards held, and the work compounded.** That's the entire promise of this setup.

Cheat sheet

A two-page foldout. Print on A3 (one side per page) or keep digital. Everything you need without opening the book.

Hooks · the 6

Hook	Event	What it does
<code>session-start.sh</code>	SessionStart	1-line recap of git state, branch, recent commits
<code>pre-tool-guard.sh</code>	PreToolUse	Blocks <code>.env</code> writes, <code>rm -rf /</code> , force pushes
<code>post-edit-format.sh</code>	PostToolUse	Formats edited files (prettier, ruff, etc.)
<code>activity-log.sh</code>	PostToolUse async	Light log of tool uses for audits
<code>coach-suggest.sh</code>	Stop	End-of-session suggestions based on what was touched
<code>extract-lesson.sh</code>	PostToolUse	After a significant commit, nudges <code>/extract-lesson</code>

Slash commands · the 8

Command	What it does
<code>/ship "<msg>"</code>	verify + commit + push in one line
<code>/audit-quick</code>	code-auditor + security-auditor in parallel
<code>/standup</code>	Daily recap: commits 24 h + WIP + suggested focus
<code>/coach / /coach-mute / /coach-on</code>	Coach mode active call / mute toggle
<code>/new-feature <slug></code>	Plan → plan-review → code → verify → audits pipeline
<code>/extract-lesson [sha]</code>	Capture 1–3 lessons from a commit into CLAUDE.md

Agent templates · the 4

Agent	Model	For
<code>code-auditor</code>	opus	Dead code, duplication, complexity, anti-patterns
<code>security-auditor</code>	opus	Secrets, XSS, injection, OWASP Top 10
<code>plan-reviewer</code>	opus	Reviews <code>PLAN_<slug>.md</code> before code (staff-engineer mode)
<code>doc-writer</code>	sonnet	READMEs, ADRs, changelog sections

Memory · where to write what

Location	Versioned?	Use for
<code>CLAUDE.md § Lessons</code>	✓	≥ 2 agents need it · cross-cutting
<code>.agents/skills/<domain>/</code>	✓	Domain rules · structured (Incorrect → Correct)
<code>.claude/agent-memory/<agent>/</code>	✓	"Don't flag this again" · per-agent
<code>~/ .claude/projects/ ... /memory/</code>	✗	Personal · this machine only · ephemeral

The decision rule, in one sentence

Slash command if it's a verb. **Sub-agent** if it's a noun with a job. **Skill** if it's a body of rules. **Hook** if it's a reflex.

Golden rules

1. `new` (and `fix` / or equivalent) must be given before every push. Unlikely, but if you're forgetful

1. `pipm verify` (or equivalent) must be green before every push. Husky pre-push if you're forgetful.
2. Plan-first when the chantier touches ≥ 3 files OR a critical module OR a database migration.
3. After a significant commit, decide: capture a lesson, or let the hook be silent.
4. Each piece of memory lives in exactly one of the four locations. No duplicates.
5. Skill > agent prompt. If a rule lives in both, the agent loses; the skill wins.
6. Run `/audit-claude-setup` once a month. Orphan skills, dormant agents, broken refs.